

```
--- Main Menu ---
1. Employee
2. Manager
3. Exit
Enter your Choice: 2

--- Manager Menu ---
1. View All Managers
2. Add Managers
3. Get Manager By ID
4. Add Employees to Manager
5. Remove Employees from Manager
6. Back
Enter Choice: 2
Enter Employee ID: 210
Enter Name: Saravana
Enter Age: 45
Enter Designation: SA
Enter Position: SA
Enter No of Employees: 10

--- Manager Menu ---
1. View All Managers
2. Add Managers
3. Get Manager By ID
4. Add Employees to Manager
5. Remove Employees from Manager
6. Back
Enter Choice: 1
ManagerName : Saravana
Manager Age : 45
Designation : SA
Position : SA
no Of Employee Under him: 10
```

```
--- Manager Menu ---
1. View All Managers
2. Add Managers
3. Get Manager By ID
4. Add Employees to Manager
5. Remove Employees from Manager
6. Back
Enter Choice: 5
Enter Manager ID: 210
Enter Number of Employees to Remove: 15
Successfull Changed

--- Manager Menu ---
1. View All Managers
2. Add Managers
3. Get Manager By ID
4. Add Employees to Manager
5. Remove Employees from Manager
6. Back
Enter Choice: 1
ManagerName : Saravana
Manager Age : 45
Designation : SA
Position : SA
no Of Employee Under him: 7

--- Manager Menu ---
1. View All Managers
2. Add Managers
3. Get Manager By ID
4. Add Employees to Manager
5. Remove Employees from Manager
6. Back
Enter Choice: 6

--- Main Menu ---
1. Employee
2. Manager
3. Exit
Enter your Choice: 1
```

```
--- Main Menu ---
1. Employee
2. Manager
3. Exit
Enter your Choice: 1

--- Employee Menu ---
1. View All Employees
2. Add Employee
3. Get Employee By ID
4. Back
Enter Choice: 2
Enter Employee ID: 1050
Enter Name: Surya
Enter Age: 21
Enter Designation: Software Engineer
Enter Position: Employee

--- Employee Menu ---
1. View All Employees
2. Add Employee
3. Get Employee By ID
4. Back
Enter Choice: 3
Enter Employee ID to Search: 1050
EmployeeName : Surya
EmployeeAge : 21
Designation : Software Engineer
Position : Employee

--- Employee Menu ---
1. View All Employees
2. Add Employee
3. Get Employee By ID
4. Back
Enter Choice:
```

ManagerModel :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day2Task.Model
{
    public class ManagerModel : EmployeeModel
    {
        public int NoOfEmployees { get; set; }

        public ManagerModel() { }
        public ManagerModel(int EmployeeID, string Name, int Age, string Designation, string
Position, string NoOfEmployees)
            : base(EmployeeID, Name, Age, Designation, Position)
        {
            this.NoOfEmployees = NoOfEmployees;
        }

        public override string ToString()
        {
            return $"ManagerName : {Name}\nManager Age : {Age} \n Designation : {Designation}
\n " +
                $"Position : {Position}\n no Of Employee Under him: {NoOfEmployees}";
        }
    }
}
```

```
CustomerModel:
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day2Task.Model
{
    public class EmployeeModel
    {
        public int EmployeeID { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public string Designation { get; set; }
        public string Position { get; set; }

        public EmployeeModel() { }
        public EmployeeModel(int EmployeeID,string Name,int Age,string Designation, string
Position)
        {
            this.EmployeeID = EmployeeID;
            this.Name = Name;
            this.Age = Age;
            this.Designation = Designation;
            this.Position = Position;
        }

        public override string ToString()
        {
            return $"EmployeeName : {Name}\nEmployeeAge : {Age} \n Designation : {Designation}
\n Position : {Position}\n";
        }
    }
}
```

```
EmployeeRepo:
using Day2Task.Model;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day2Task.Repo
{
    public class EmployeeRepo : IEmployeeRepo
    {
        List<EmployeeModel> employeeList = new List<EmployeeModel>();
        public void AddEmployee(EmployeeModel employee)
        {
            employeeList.Add(employee);
        }

        public void GetALLEmployees()
        {
            foreach (EmployeeModel employee in employeeList)
            {
                Console.WriteLine(employee.ToString());
            }
        }

        public void GetEmployeeByID(int EmployeeID)
        {
            var employee = employeeList.Where(e => e.EmployeeID ==
EmployeeID).FirstOrDefault();
            if (employee != null)
            {
                Console.WriteLine(employee.ToString());
            }else
            {
                Console.WriteLine("Not Found");
            }
        }
    }
}
```

IEmployeRepo:

```
using Day2Task.Model;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace Day2Task.Repo
{
    public interface IEmployeeRepo
    {
        void GetALLEmployees();

        void GetEmployeeByID(int EmployeeID);

        void AddEmployee(EmployeeModel employee);
    }
}
```

IManagerRepo:

```
using Day2Task.Model;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace Day2Task.Repo
{
    internal interface IManagerRepo
    {
        void GetAllManagers();
        void GetManagerByID(int EmployeeID);

        void AddManager(ManagerModel manager);

        void AddEmployeeToManager(int ManagerID, int Count);

        void RemoveEmployeeFromManager(int ManagerID, int Count);
    }
}
```

```
ManagerRepo:
using Day2Task.Model;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day2Task.Repo
{
    internal class ManagerRepo : IManagerRepo
    {
        List<ManagerModel> managerlist = new List<ManagerModel>();

        public void GetAllManagers()
        {
            foreach (EmployeeModel manager in managerlist)
            {
                Console.WriteLine(manager.ToString());
            }
        }

        public void GetManagerByID(int EmployeeID)
        {
            var manager = managerlist.Where(e => e.EmployeeID == EmployeeID).FirstOrDefault();
            if (manager != null)
            {
                Console.WriteLine(manager.ToString());
            }
            else
            {
                Console.WriteLine("Not Found");
            }
        }

        public void AddManager(ManagerModel manager)
        {
            managerlist.Add(manager);
        }

        public void AddEmployeeToManager(int ManagerID, int Count)
        {
            var manager = managerlist.Where(e => e.EmployeeID == ManagerID).FirstOrDefault();
```



```
        if (manager != null)
        {
            manager.NoOfEmployees += Count;
            Console.WriteLine("Successfull Changed");
        }
        else
        {
            Console.WriteLine("Manager Not Found");
        }
    }
    public void RemoveEmployeeFromManager(int ManagerID, int Count)
    {
        var manager = managerlist.Where(e => e.EmployeeID == ManagerID).FirstOrDefault();
        if (manager != null)
        {
            manager.NoOfEmployees -= Count;
            Console.WriteLine("Successfull Changed");
        }
        else
        {
            Console.WriteLine("Manager Not Found");
        }
    }
}
```

Program:

```
using System;
```

```
using Day2Task.Model;
```

```
using Day2Task.Repo;
```

```
internal class Program
```

```
{
```

```
    private static void Main(string[] args)
```

```
    {
```

```
        IEmployeeRepo Emprepo = new EmployeeRepo();
```

```
        IManagerRepo Managerrepo = new ManagerRepo();
```

```
        while (true)
```

```
        {
```

```
            Console.WriteLine("\n--- Main Menu ---");
```

```
            Console.WriteLine("1. Employee");
```

```
            Console.WriteLine("2. Manager");
```

```
            Console.WriteLine("3. Exit");
```

```
            Console.Write("Enter your Choice: ");
```

```
            int ch;
```

```
            if (!int.TryParse(Console.ReadLine(), out ch))
```

```
            {
```

```
                Console.WriteLine("Invalid input!");
```

```
                continue;
```

```
            }
```

```
            switch (ch)
```

```
            {
```

```
                case 1:
```

```
                    EmployeeMenu(Emprepo);
```

```
                    break;
```

```
                case 2:
```

```
                    ManagerMenu(Managerrepo);
```

```
                    break;
```

```
                case 3:
```

```
                    return;
```

```
                default:
```

```
                    Console.WriteLine("Invalid Choice");
```

```
                    break;
```

```
            }
```

```
        }
```

```
    }
```

```
private static void EmployeeMenu(IEmployeeRepo Emprepo)
{
    int empChoice = 0;
    do
    {
        Console.WriteLine("\n--- Employee Menu ---");
        Console.WriteLine("1. View All Employees");
        Console.WriteLine("2. Add Employee");
        Console.WriteLine("3. Get Employee By ID");
        Console.WriteLine("4. Back");
        Console.Write("Enter Choice: ");
        if (!int.TryParse(Console.ReadLine(), out empChoice))
        {
            Console.WriteLine("Invalid input!");
            continue;
        }
        switch (empChoice)
        {
            case 1:
                Emprepo.GetAllEmployees();
                break;
            case 2:
                EmployeeModel newEmp = new EmployeeModel();
                Console.Write("Enter Employee ID: ");
                newEmp.EmployeeID = int.Parse(Console.ReadLine());
                Console.Write("Enter Name: ");
                newEmp.Name = Console.ReadLine();
                Console.Write("Enter Age: ");
                newEmp.Age = int.Parse(Console.ReadLine());
                Console.Write("Enter Designation: ");
                newEmp.Designation = Console.ReadLine();
                Console.Write("Enter Position: ");
                newEmp.Position = Console.ReadLine();
                Emprepo.AddEmployee(newEmp);
                break;
            case 3:
                Console.Write("Enter Employee ID to Search: ");
                int searchId = int.Parse(Console.ReadLine());
                Emprepo.GetEmployeeById(searchId);
                break;
            case 4:
```

```
        break;
    default:
        Console.WriteLine("Invalid Choice");
        break;
    }
} while (empChoice != 4);
}

private static void ManagerMenu(IManagerRepo Managerrepo)
{
    int managerChoice = 0;
    do
    {
        Console.WriteLine("\n--- Manager Menu ---");
        Console.WriteLine("1. View All Managers");
        Console.WriteLine("2. Add Managers");
        Console.WriteLine("3. Get Manager By ID");
        Console.WriteLine("4. Add Employees to Manager");
        Console.WriteLine("5. Remove Employees from Manager");
        Console.WriteLine("6. Back");
        Console.Write("Enter Choice: ");
        if (!int.TryParse(Console.ReadLine(), out managerChoice))
        {
            Console.WriteLine("Invalid input!");
            continue;
        }
        switch (managerChoice)
        {
            case 1:
                Managerrepo.GetAllManagers();
                break;
            case 2:
                ManagerModel newmang = new ManagerModel();
                Console.Write("Enter Employee ID: ");
                newmang.EmployeeID = int.Parse(Console.ReadLine());
                Console.Write("Enter Name: ");
                newmang.Name = Console.ReadLine();
                Console.Write("Enter Age: ");
                newmang.Age = int.Parse(Console.ReadLine());
                Console.Write("Enter Designation: ");
                newmang.Designation = Console.ReadLine();
                Console.Write("Enter Position: ");
                newmang.Position = Console.ReadLine();
            
```

```
        Console.WriteLine("Enter No of Employees: ");
        newmang.NoOfEmployees = int.Parse(Console.ReadLine());
        Managerrepo.AddManager(newmang);
        break;
    case 3:
        Console.WriteLine("Enter Manager ID to Search: ");
        int manId = int.Parse(Console.ReadLine());
        Managerrepo.GetManagerById(manId);
        break;
    case 4:
        Console.WriteLine("Enter Manager ID: ");
        int addManId = int.Parse(Console.ReadLine());
        Console.WriteLine("Enter Number of Employees to Add: ");
        int addCount = int.Parse(Console.ReadLine());
        Managerrepo.AddEmployeeToManager(addManId, addCount);
        break;
    case 5:
        Console.WriteLine("Enter Manager ID: ");
        int rmManId = int.Parse(Console.ReadLine());
        Console.WriteLine("Enter Number of Employees to Remove: ");
        int rmCount = int.Parse(Console.ReadLine());
        Managerrepo.RemoveEmployeeFromManager(rmManId, rmCount);
        break;
    case 6:
        break;
    default:
        Console.WriteLine("Invalid Choice");
        break;
    }
} while (managerChoice != 6);
}
```